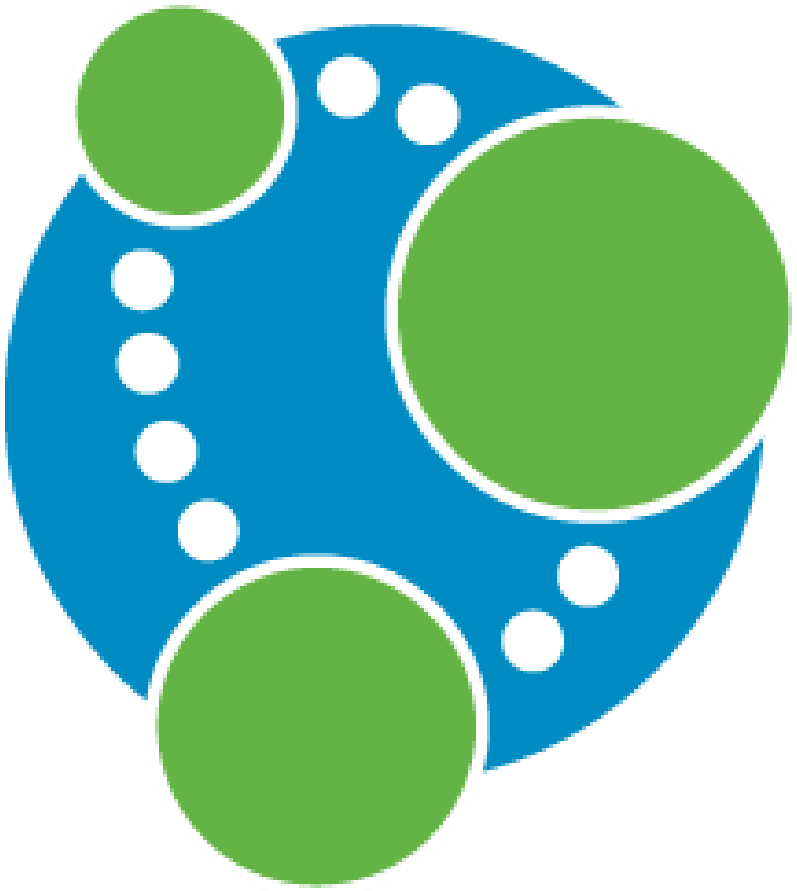
Neo4j

SECTION4

ENG. ASMAA FATHY

What is Neo4j ?

- Neo4j is a NoSQL database. It is highly scalable and schema-free. It's world most popular graph database management system.
- Neo4j is implemented in Java language and it can be accessed by other language using Cypher Query Language (CQL) through a transactional HTTP endpoint.
- Neo4j is a way faster than traditional databases.



Neo4j Working

- Neo4j stores and displays data in the form of graph. In Neo4j, data is represented by nodes and relationships between those nodes.
- Neo4j is best for storing data that has many interconnecting relationships that's why graph databases like Neo4j has an advantage and much better at dealing with relational data than relational databases are.
- The graph model doesn't usually require a predefined schema. So there is no need to create the database structure before you load the data (like you do in a relational database). In Neo4j, the data is the structure. Neo4j is a "schema-optional" DBMS.
- In Neo4j, no need to set up primary key/foreign key constraints to predetermine which fields can have a relationship, and to which data. You just have to define the relationships between the nodes you need.

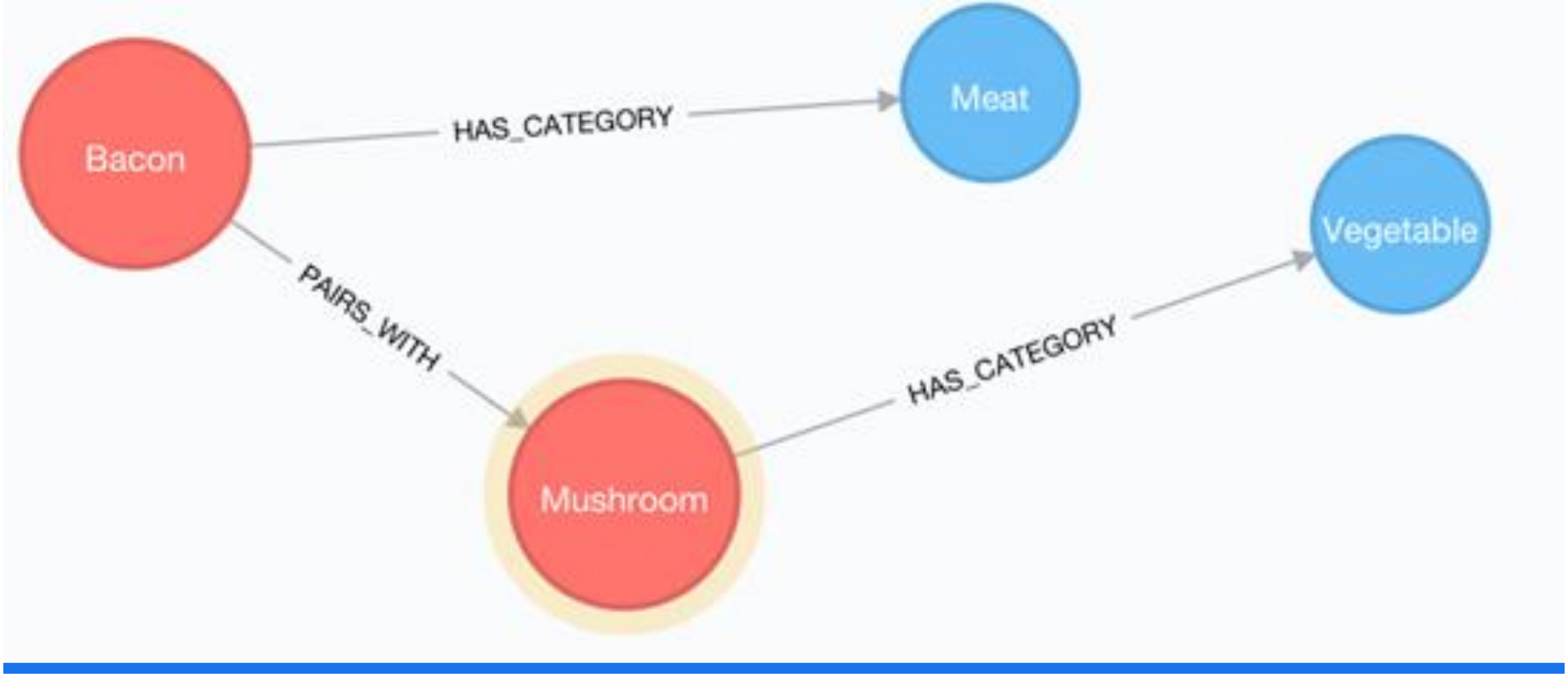
Neo4j Installation

https://www.youtube.com/watch?v=uUkBjrrBXN4

https://neo4j.com/download/

Neo4j Data Model

- Neo4j Database follows the Property Graph Model for storing and managing its data. Neo4j is a graph database which contains the following features of Property Graph Model.
  - ❑ The Graph model contains Nodes, Relationships and Properties which specifies data and its operation.
  - ❑ Properties are key-value pairs.
  - ❑ Nodes are represented using circle and Relationships are represented using arrow keys. Relationship specifies the relation between two nodes.
  - ❑ There are two types of relationships between nodes according to their directions: Unidirectional and Bidirectional
  - ❑ Each Relationship contains two nodes: "Start Node" or "From Node" and "To Node" or "End Node".
  - ❑ Both Nodes and Relationships contain properties.
- Relationships should be directional in Property Graph Data Mode. If you create a relationship without a direction, it will through an error message.



There are three main building block of a GraphDB Data model:

- Nodes
- Relationship
- Properties

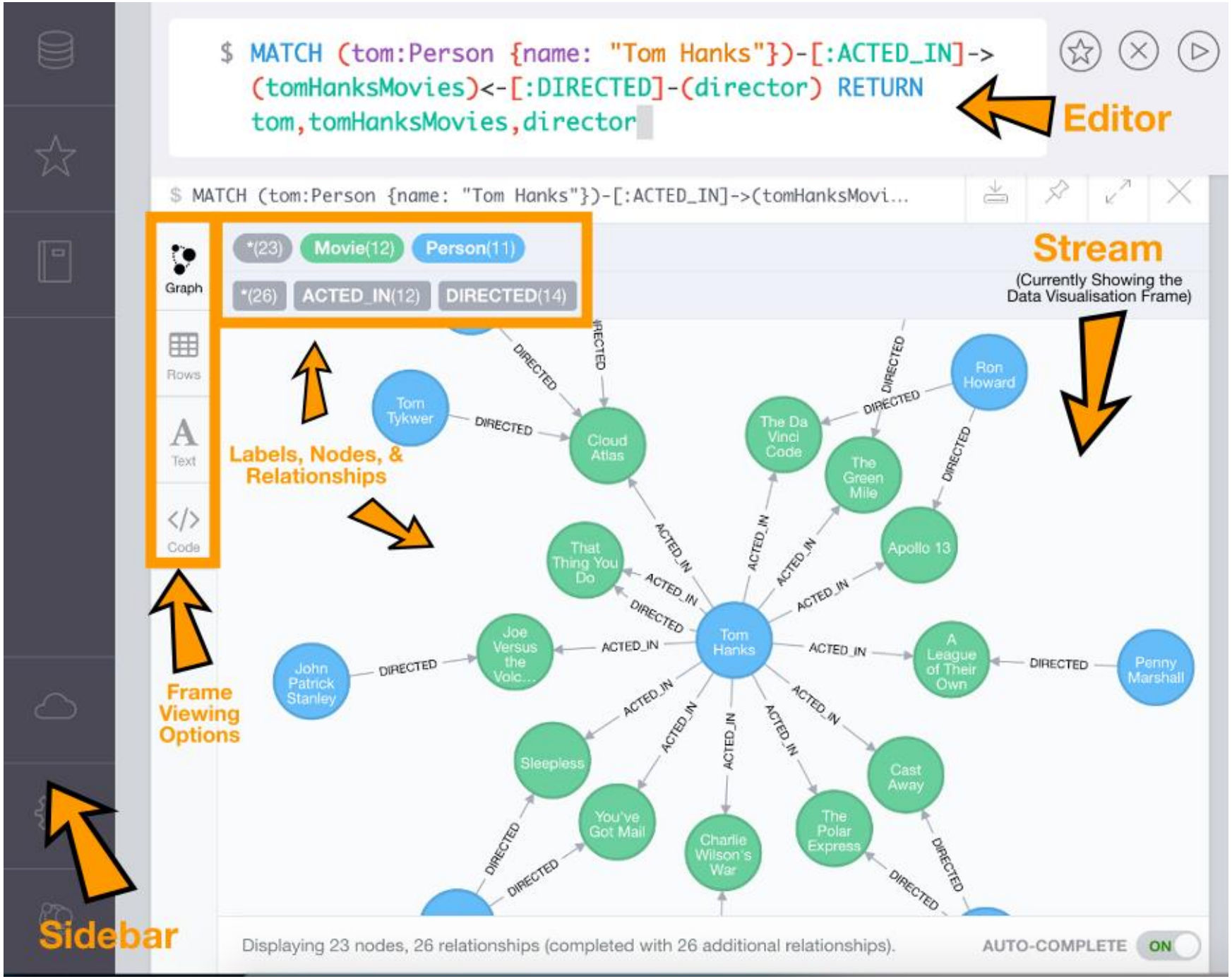
What is Neo4j CQL ?

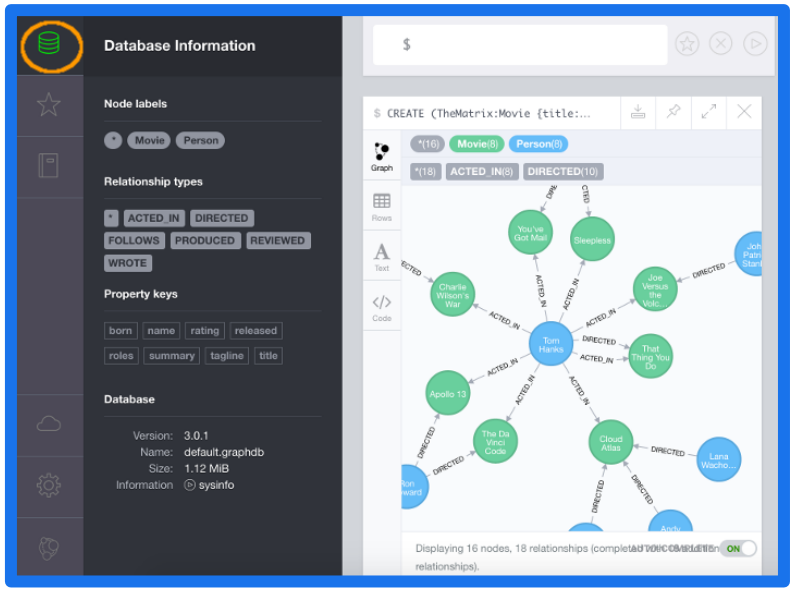
CQL stands for Cypher Query Language. It is a query language for Neo4j just like SQL is a query language for Oracle or MySQL.

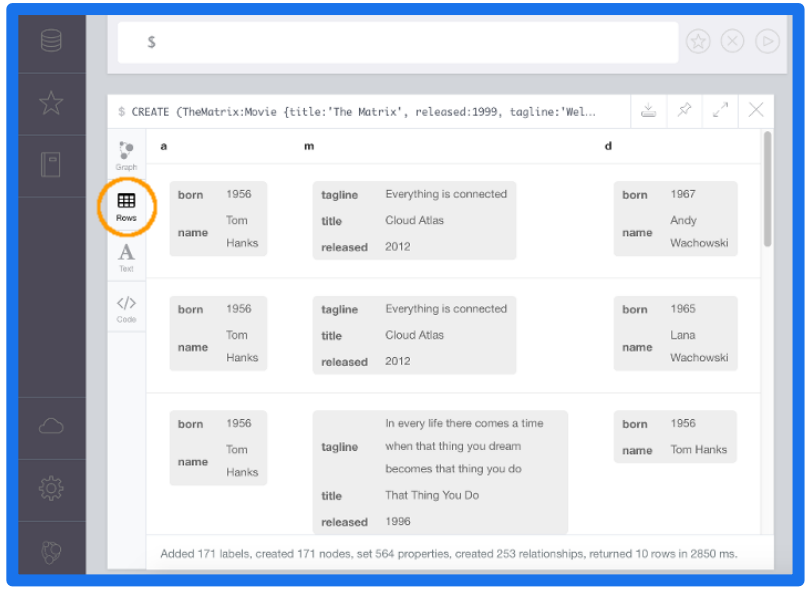
Is a declarative pattern-matching language.

The syntax of CQL is same like SQL syntax.

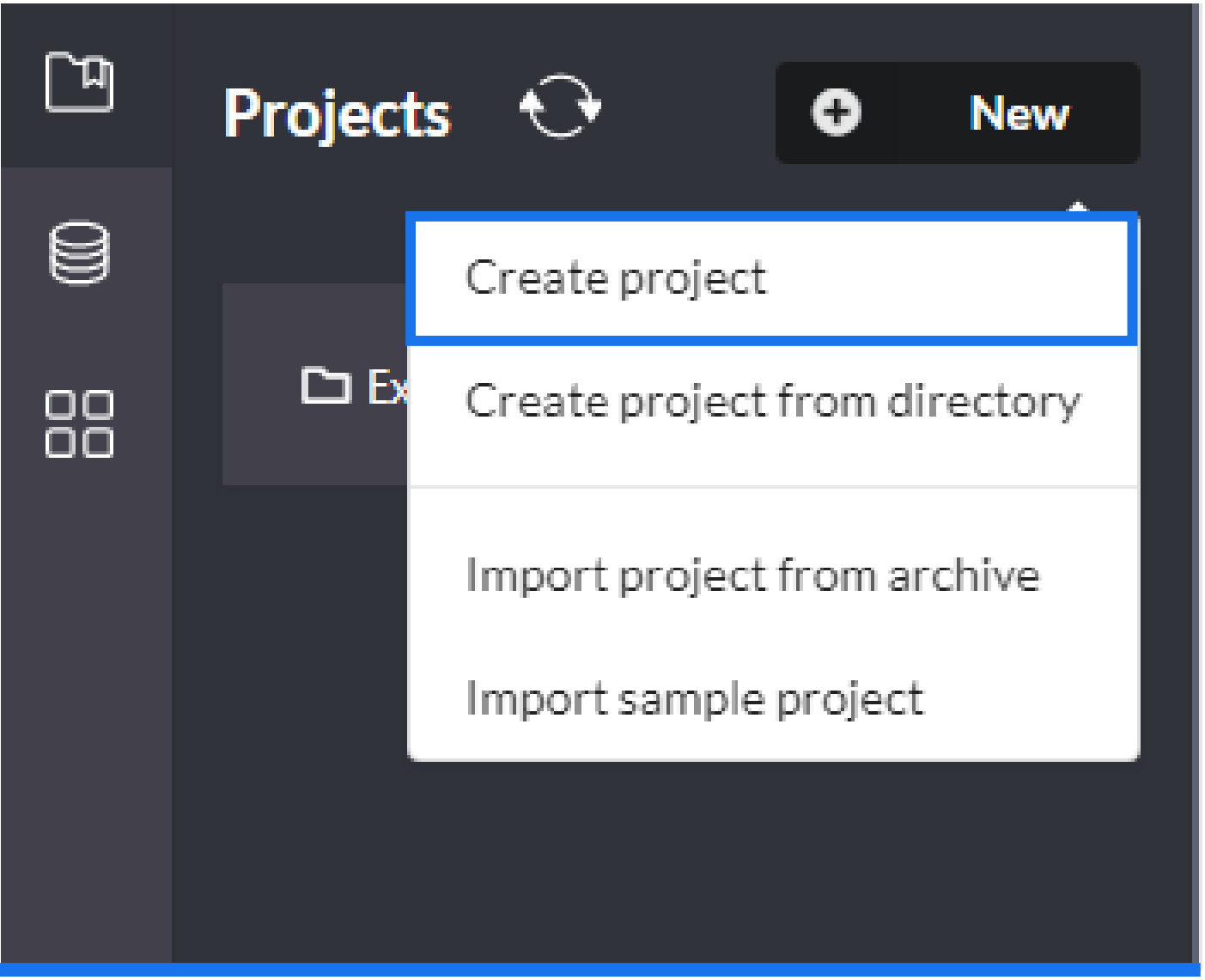
Neo4j Browser Interface







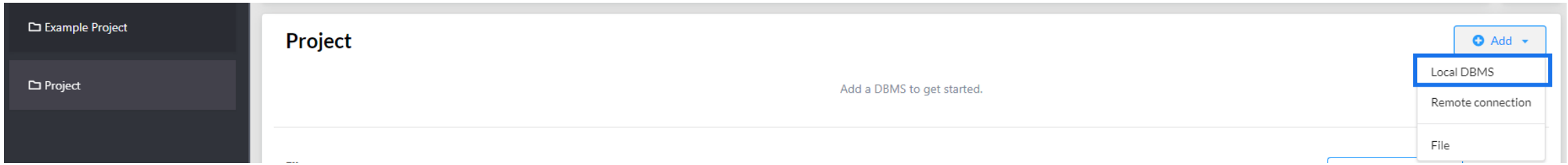
1. Editor: This is where you enter queries and commands.
2. Stream: This is where the results of your queries appear.
3. Labels, Nodes, & Relationships: These represent the data in the database.
4. Sidebar : The Sidebar has various options, such as viewing the details of your database, viewing/changing Neo4j Browser Settings, viewing Neo4j Documentation, and more.
5. Frame Viewing Options : This enables you to view the data in different ways. Clicking on Rows for example, will display the nodes and relationships in rows.



Create Project

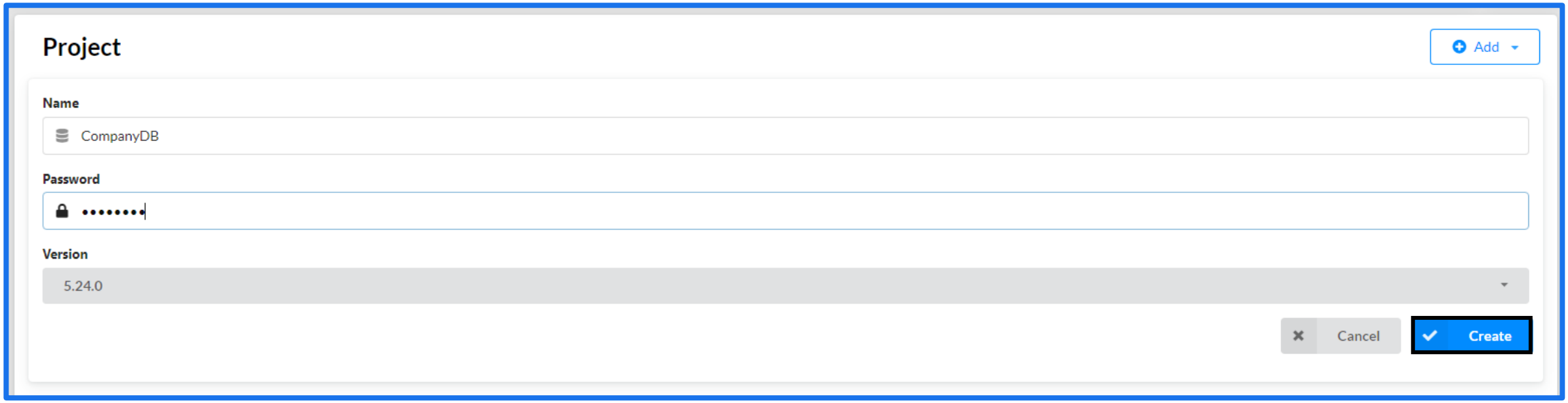
- Project in Neo4j Desktop is a way to organize databases and queries within a single working environment.

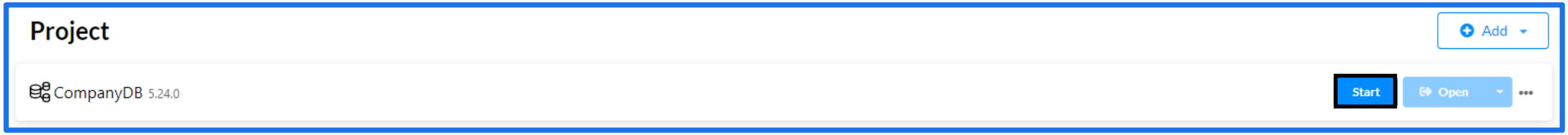
Create DBMS


Example Project
Project
Project
Add
Add a DBMS to get started.
Local DBMS
Remote connection
File

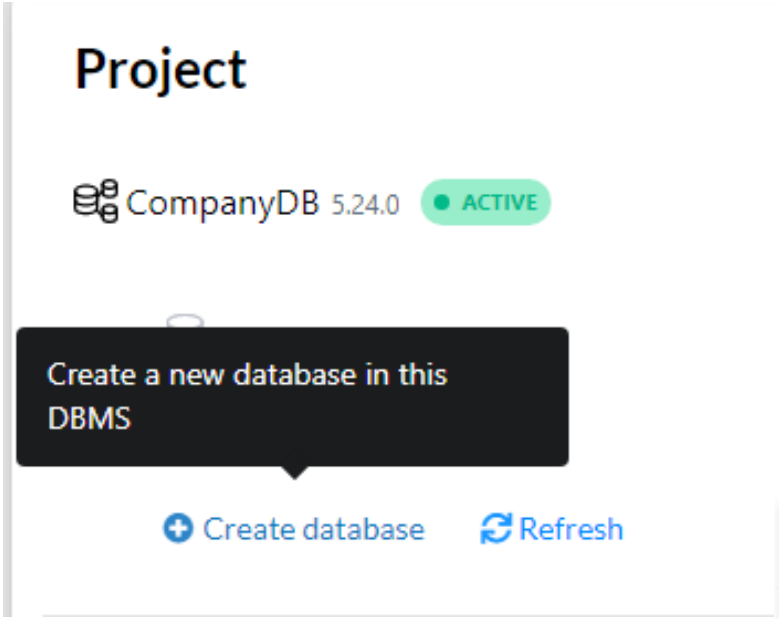
Configure DBMS Settings

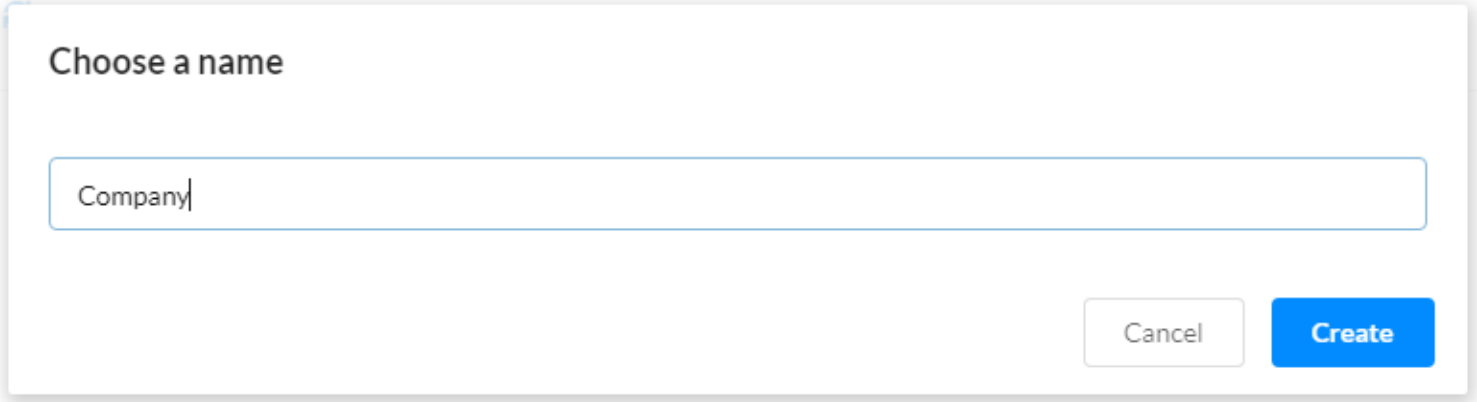
1. Enter a DBMS name (e.g., MyGraphDB).
2. Enter the neo4j user password.
3. Choose the Neo4j latest available version.
4. Start DBMS.



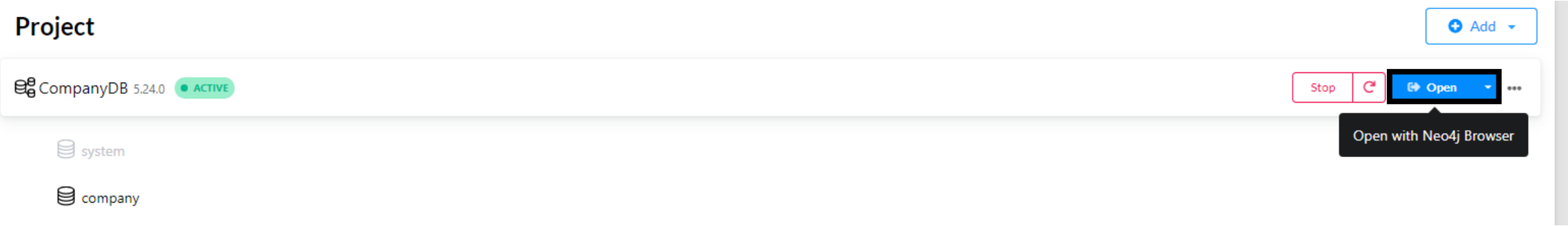


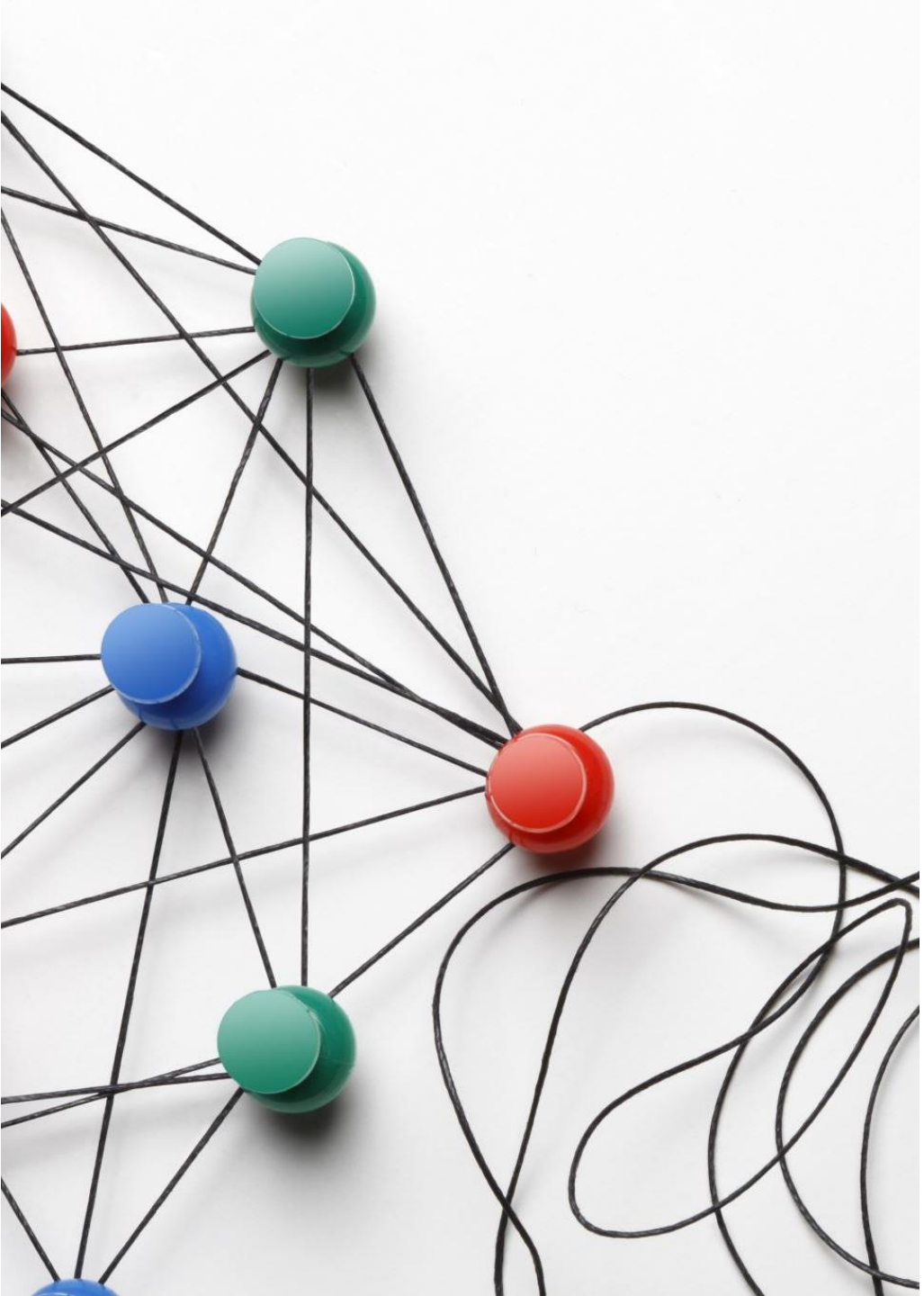
Create Database in DBMS





Open Neo4j Browser

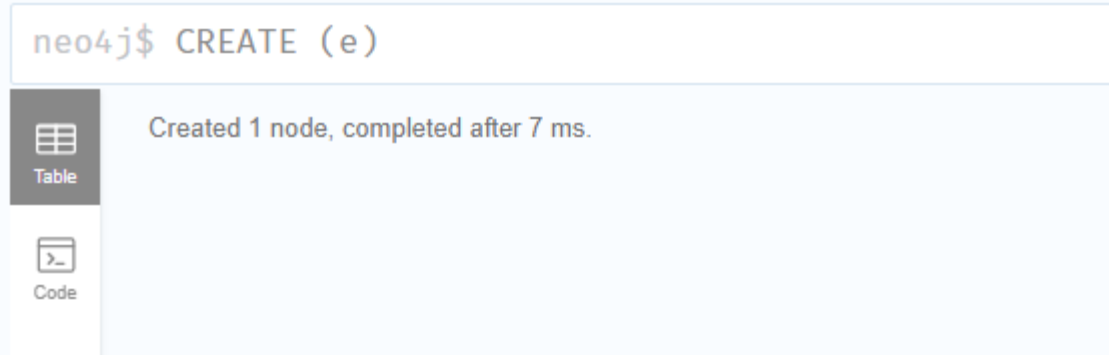




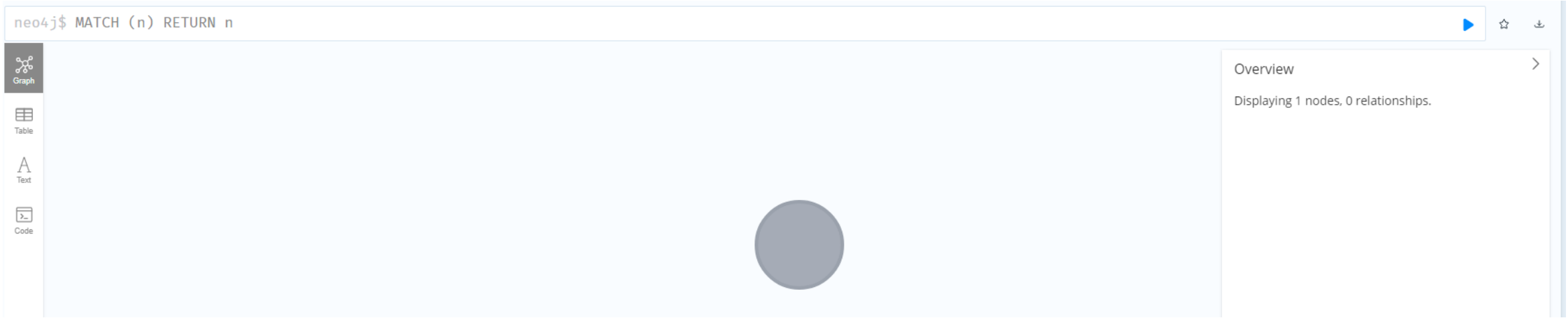
Neo4j Create Nodes

- Node is a data or record in a graph database.
- In Neo4j, CREATE statement is used to create a node. You can create the following things by using CREATE statement:
  - ✓ Create a single node
  - ✓ Create multiple nodes
  - ✓ Create a node with a label
  - ✓ Create a node with multiple labels
  - ✓ Create a node with properties
  - ✓ Returning the created node

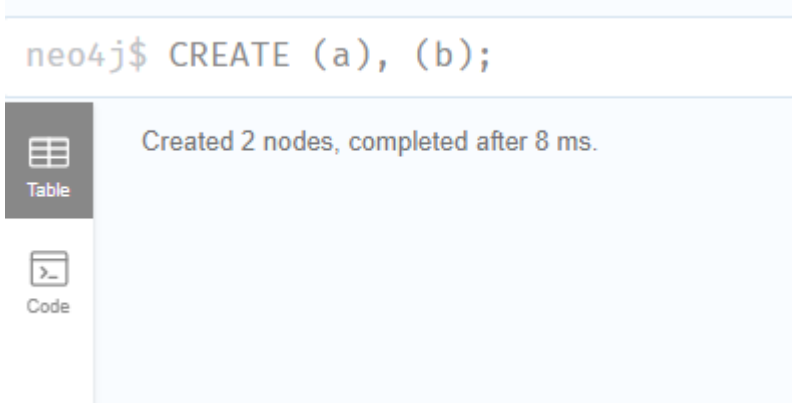
Create a Single Node



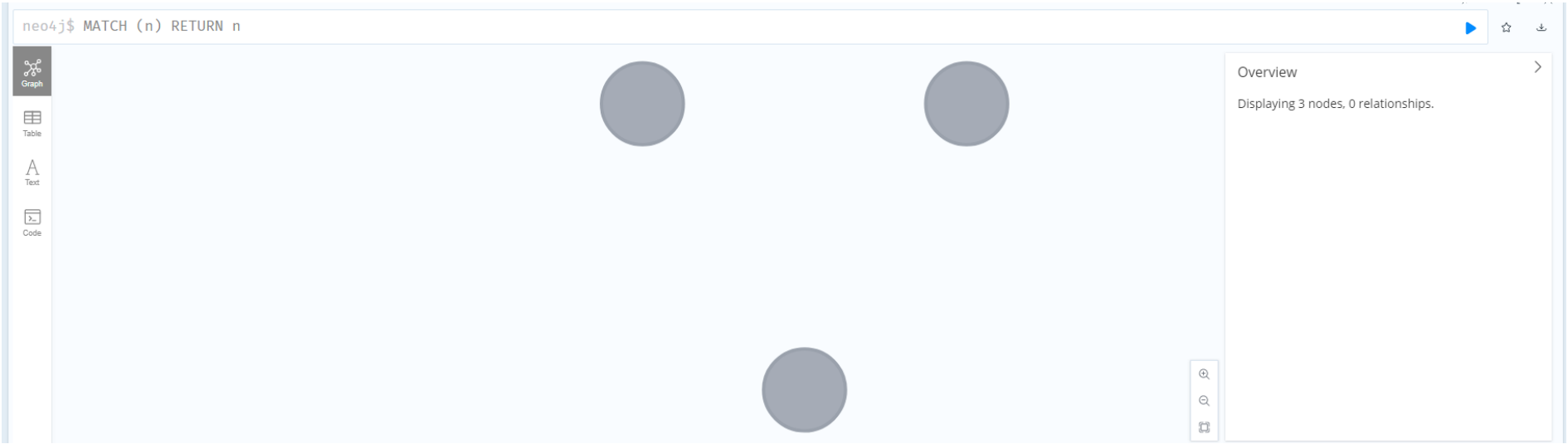
- Syntax: CREATE (node_name);
- Note: You can add or ignore semicolon (;). It is optional.
- Execute the following code to verify the creation of the node type:



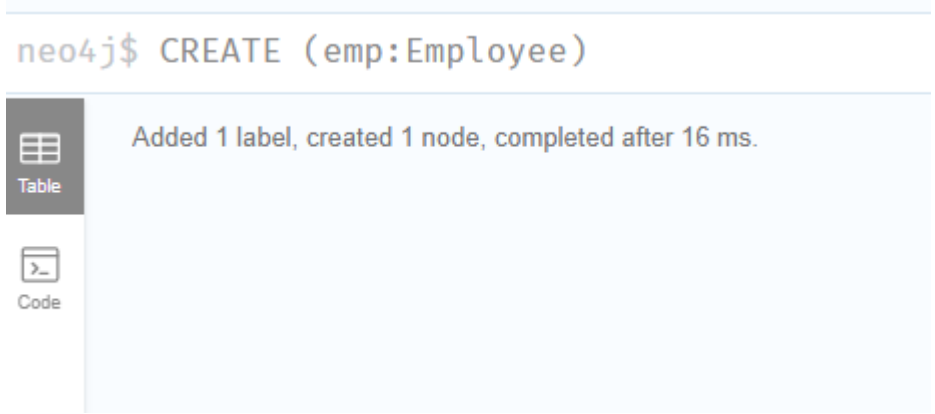
Create Multiple Nodes



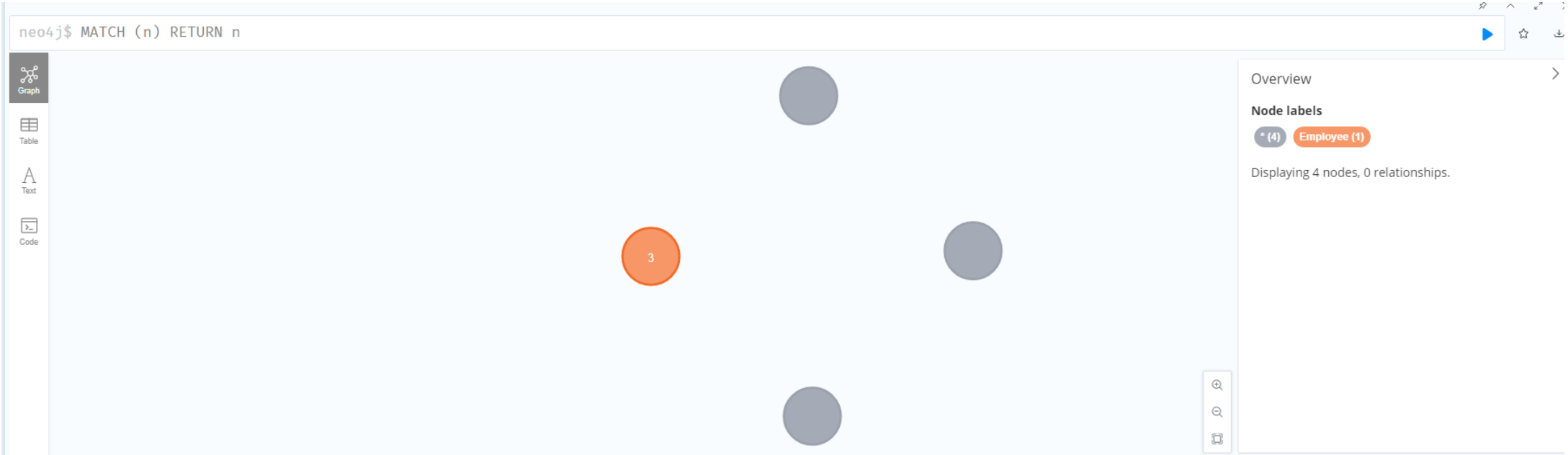
- To create multiple nodes in Neo4j, use CREATE statement with the name of nodes separated by a comma.
- Syntax: CREATE (node1),(node2), (node1),???..
- Verification:



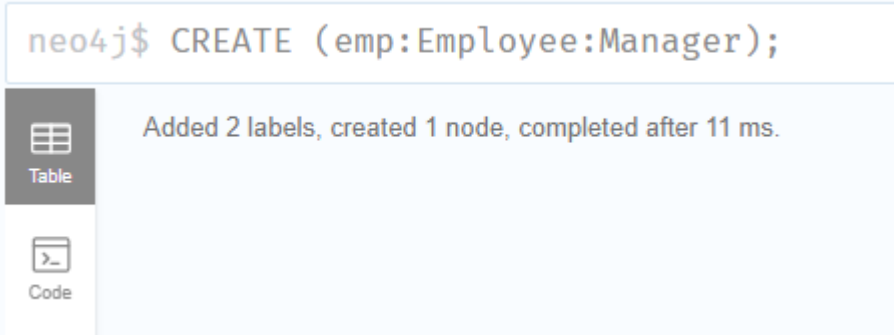
Create a node with a label



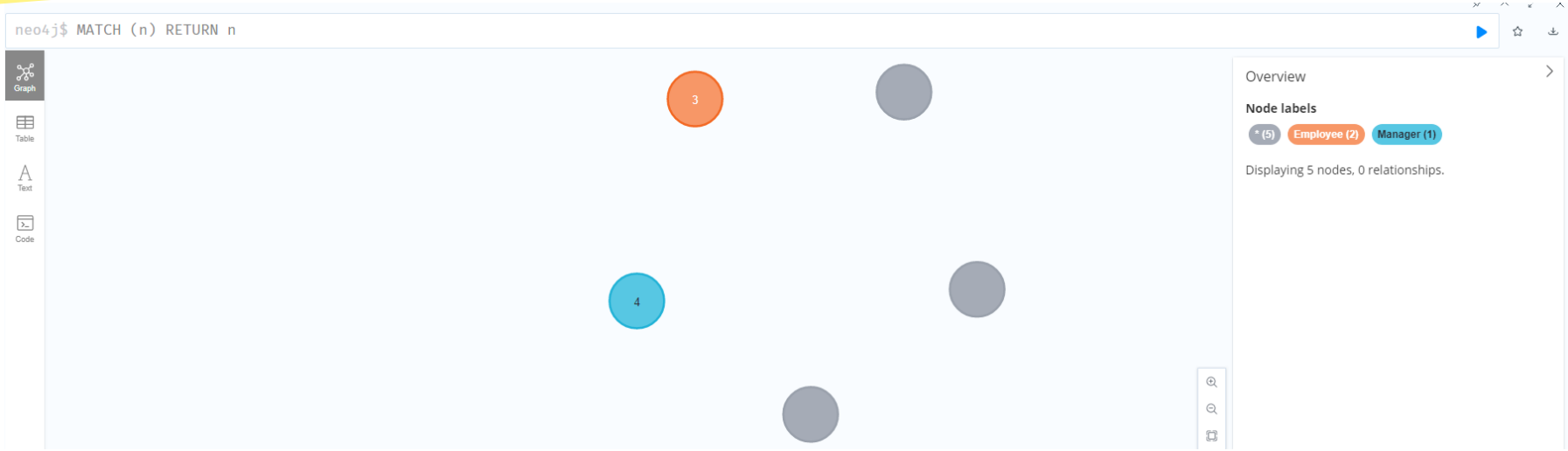
- Syntax: CREATE (node:label)
- Verification:



Create a Node with Multiple Labels



- To create multiple labels with a single node, you have to specify the labels for the node by separating them with a colon " : ".
- Syntax: CREATE (node:label1:label2:. . . . labeln)
- Verification:



Create Node with Properties

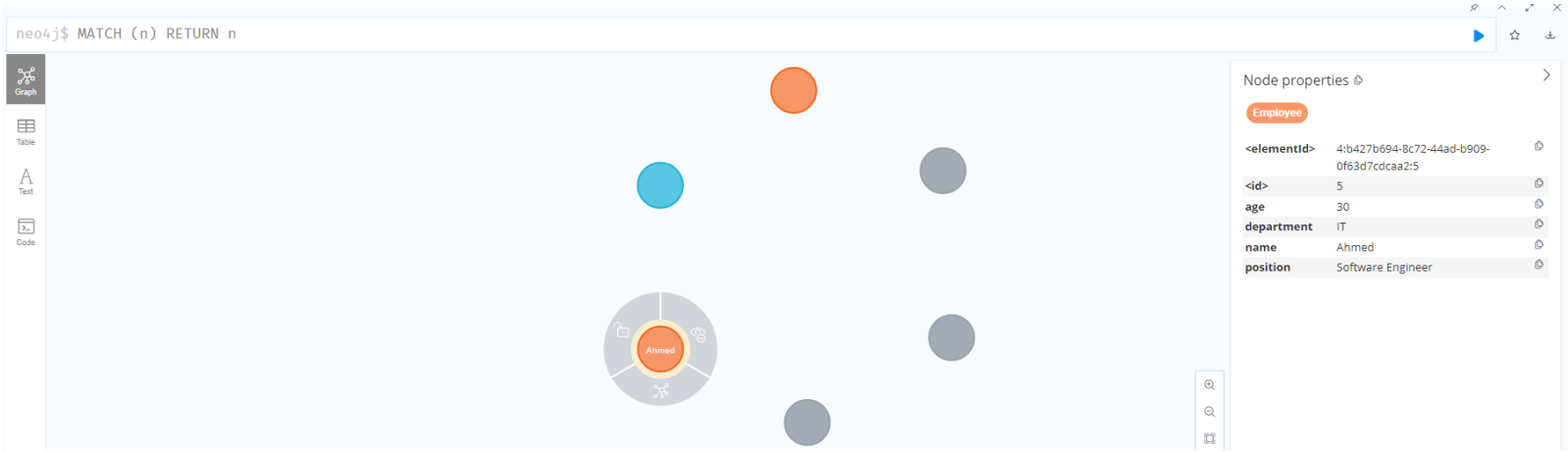
```
neo4j$ CREATE (emp:Employee {name: "Ahmed", age: 30, position: "Software Engineer", department: "IT"});
```

Table

Code

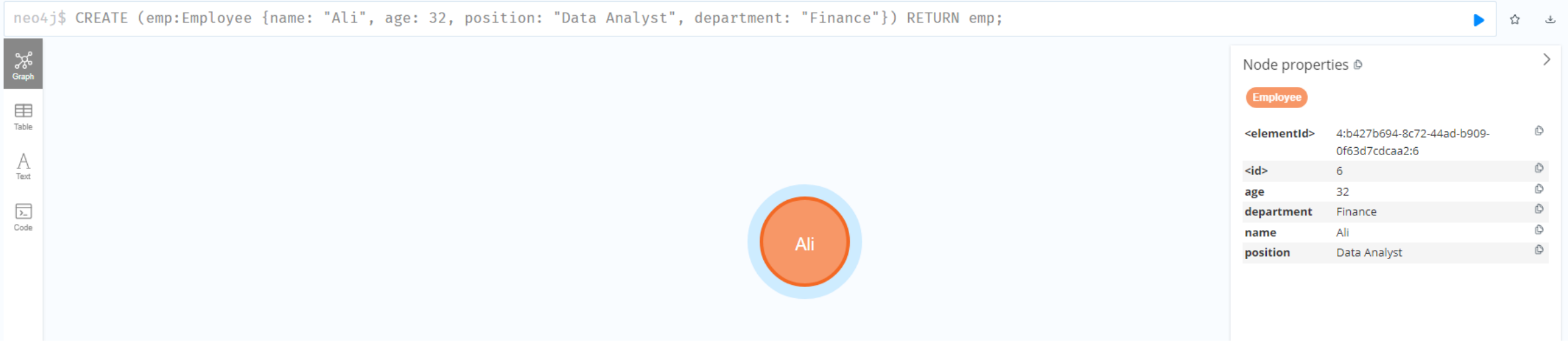
Added 1 label, created 1 node, set 4 properties, completed after 19 ms.

- In Neo4j, properties are the key-value pairs which are used by nodes to store data. CREATE statement is used to create node with properties, you just have to specify these properties separated by commas within the curly braces "{ }".
- Syntax: CREATE (node:label { key1: value, key2: value, ... }).
- Verification:



Returning Created Node

- MATCH (n) RETURN n command is used to view the created nodes. This query returns all the existing nodes in the database.
- But if you want to return the newly created node use the RETURN command with CREATE command:
- Syntax: CREATE (Node:Label{properties. . . }) RETURN Node



Delete Node

```
neo4j$ MATCH (e:Employee {name: "Ahmed"}) DELETE e
```

Table

Code

Deleted 2 nodes, completed after 32 ms.

- In Neo4j, DELETE statement is always used with MATCH statement to delete whatever data is matched.
- Syntax: MATCH (Node:Label {key1:value}) DELETE Node

Delete Multiple Nodes

- You can delete multiple nodes by using MATCH and DELETE commands in a single statement.
- You just must put the different nodes separated by a column.

```
neo4j$ MATCH (e:Employee {name: "Ahmed"}), (c:Client {name: "TechCorp"}) DELETE e,c
```

Table

Deleted 2 nodes, completed after 43 ms.

Delete All Nodes

```
neo4j$ MATCH (n) DELETE n;
```

Table

Code

Deleted 14 nodes, completed after 9 ms.

- To delete all nodes from the database, don't use any filter criteria.

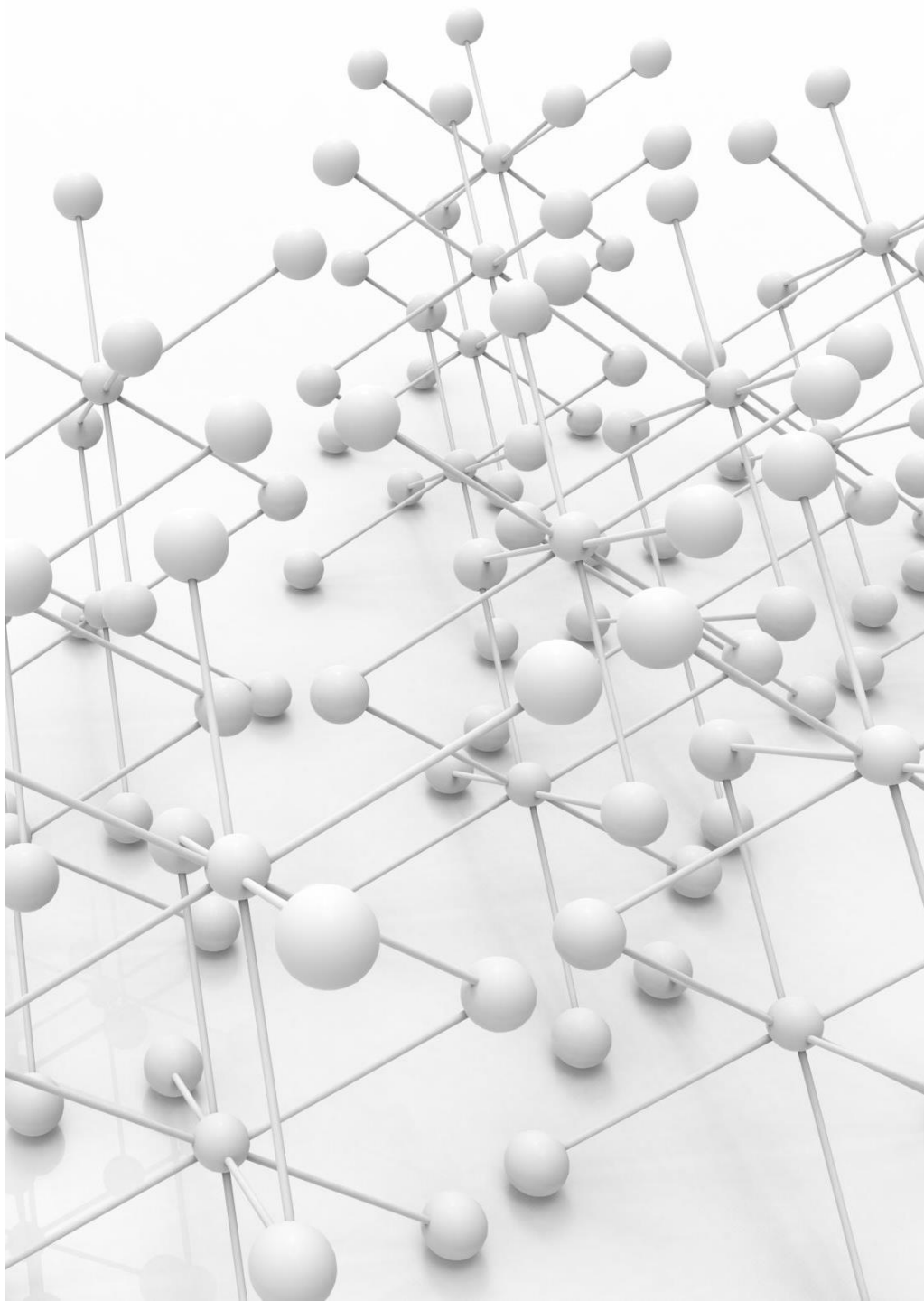
Delete Node and All Relationships Related to That Node

- Use the DETACH DELETE statement:

```
neo4j$ MATCH (e:Employee {name: "Laila"}) DETACH DELETE e;
```

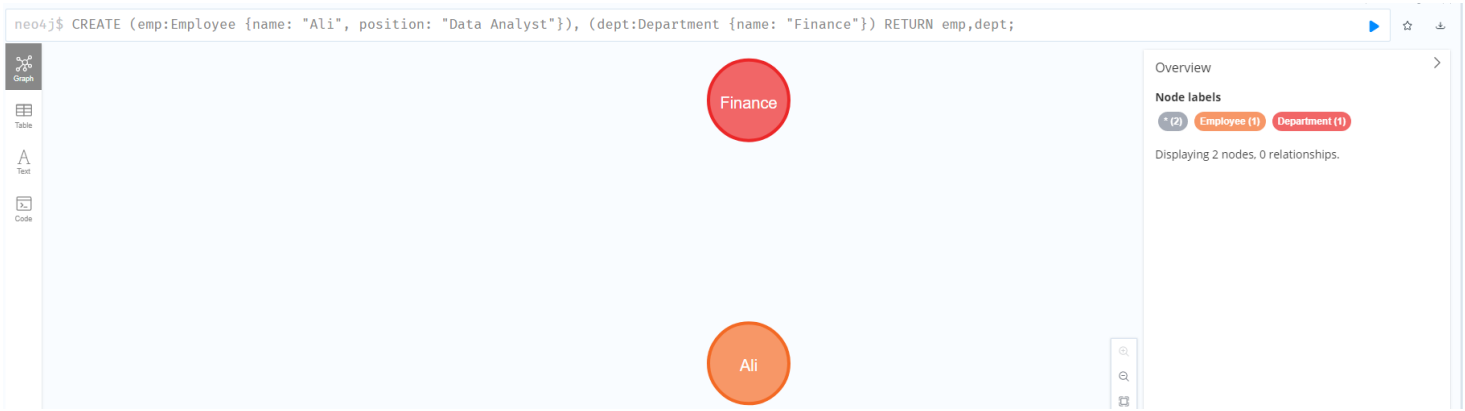
Create Relationship

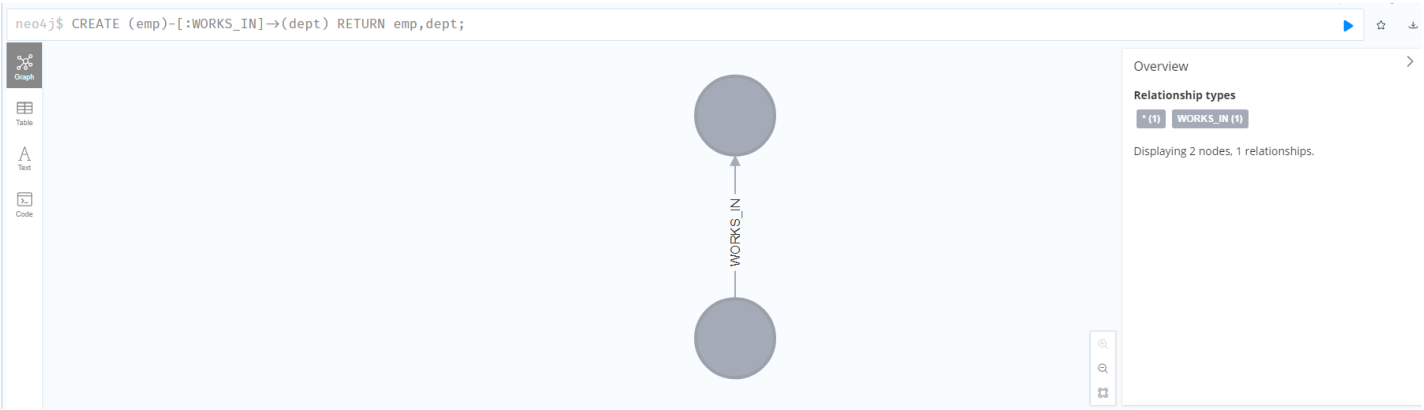
- CREATE statement is used to create relationship between nodes. These relationships define direction, type and form patterns of the data.
- It defines mainly three things:
  - ✓ Creating Relationships
  - ✓ Creating Relationships between existing nodes
  - ✓ Creating relationships with label and properties



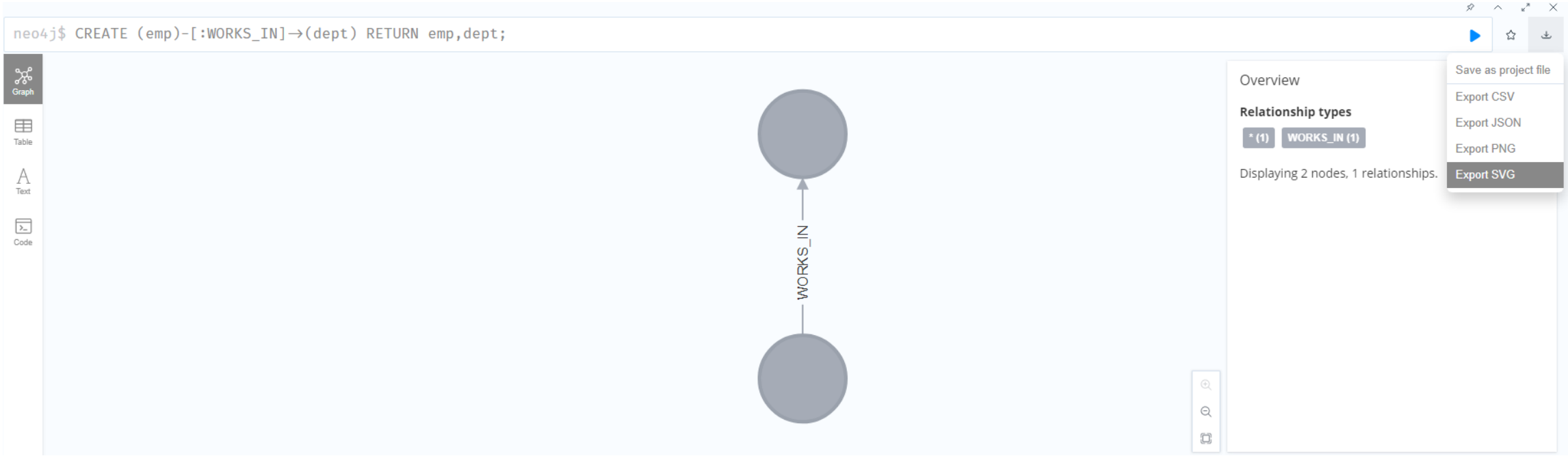
Creating Relationship

- While creating a relationship, relationship should be specified within square braces "[ ]", depending on the direction of the relationship it is placed between hyphen " - " and arrow " ? " as shown in the following syntax.
- Syntax: CREATE (node1)-[:RelationshipType]->(node2)





You can also download (export) the graph in which format you want to save it. Click on the download button, see the example:



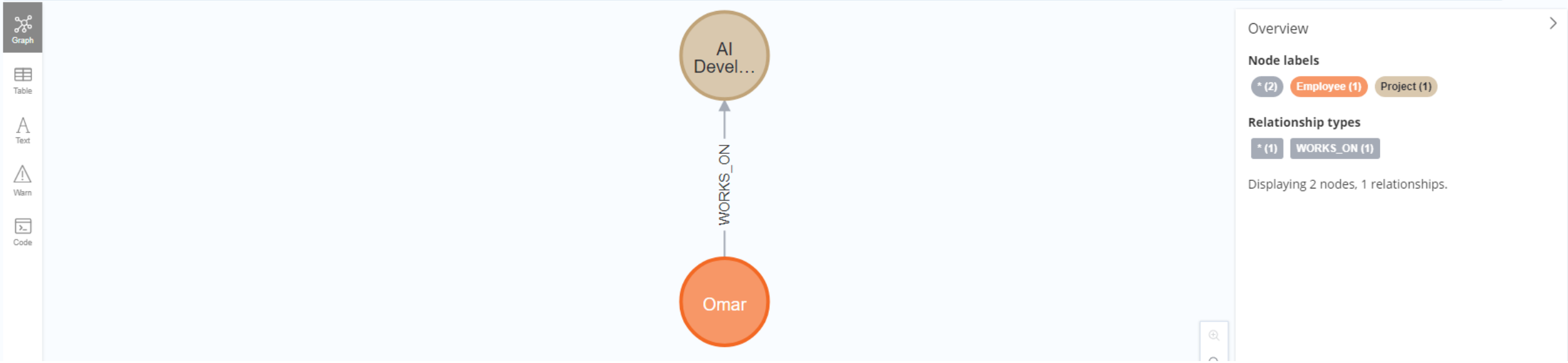
Create a Relationship between existing Nodes

- MATCH statement is used to create relationship between the existing Nodes.
- Syntax:

```
MATCH (a:LabeofNode1), (b:LabeofNode2)
WHERE a.name = "nameofnode1" AND b.name = " nameofnode2"
CREATE (a)-[: Relation]->(b)
RETURN a,b
```

```
neo4j$ CREATE (emp:Employee {name: "Omar", position: "Software Engineer"}), (proj:Project {name: "AI Development"});
```

```
neo4j$ MATCH (emp:Employee), (proj:Project) WHERE emp.name = "Omar" AND proj.name = "AI Development" CREATE (emp)-[:WORKS_ON]→(proj) RETURN emp, proj;
```



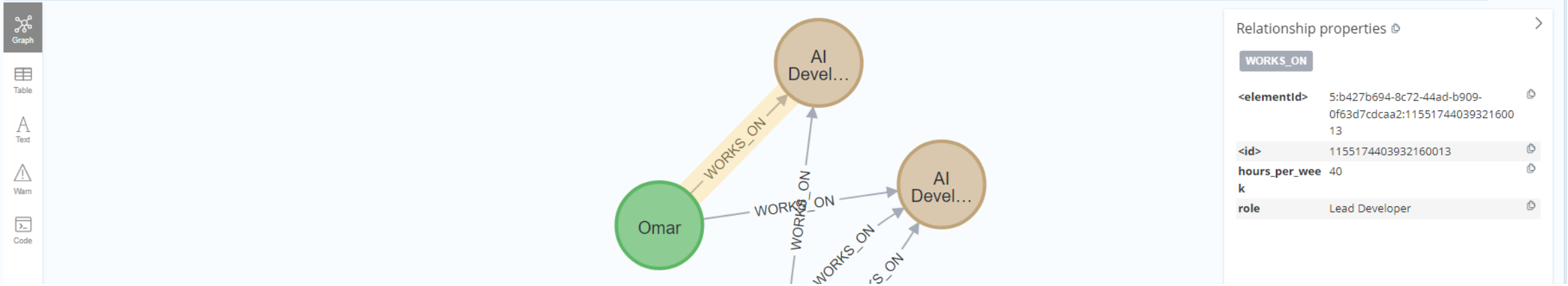
Create a Relationship with Label and Properties

- CREATE statement is used to create a relationship with label and properties.
- Syntax: CREATE (node1)-[label:Rel_Type {key1:value1, key2:value2, . . . n}]-> (node2)

```
neo4j$ CREATE (emp:Employee:Developer {name: "Omar", position: "Software Engineer", experience: 5});
```

```
neo4j$ CREATE (proj:Project {name: "AI Development", budget: 50000});
```

```
MATCH (emp:Employee), (proj:Project)
WHERE emp.name = "Omar" AND proj.name = "AI Development"
CREATE (emp)-[:WORKS_ON {role: "Lead Developer", hours_per_week: 40}]→(proj)
RETURN emp, proj;
```



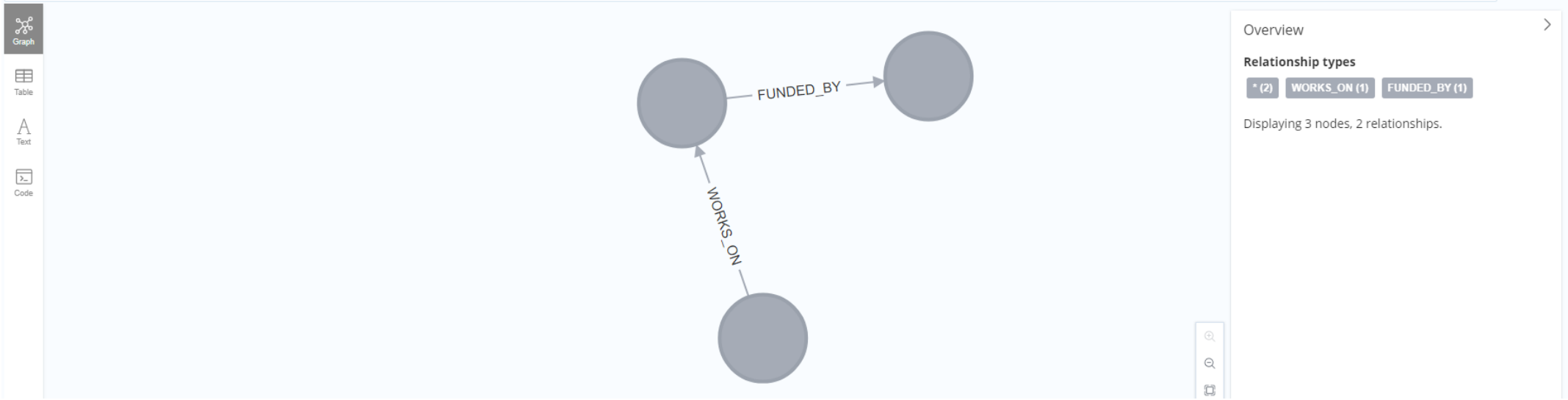
Creating a complete path

- In Neo4j, CREATE statement is used to create a path. A path is formed using continuous relationship.
- Syntax:

  CREATE p = (Node1 {properties})-[:Relationship_Type]->(Node2 {properties})[:Relationship_Type]->(Node3 {properties})
  RETURN p

```
neo4j$ CREATE (emp:Employee {name: "Ahmed", position: "Software Engineer", experience: 5})
neo4j$ CREATE (proj:Project {name: "AI Development", budget: 50000})
neo4j$ CREATE (client:Client {name: "TechCorp", industry: "AI Solutions"})
```

```
neo4j$ CREATE p = (emp)-[:WORKS_ON {role: "Lead Developer", hours_per_week: 40}]→ (proj)-[:FUNDED_BY {amount: 50000}]→(client) RETURN p;
```



Neo4j Delete a Relationship

```
neo4j$ MATCH (emp)-[r:WORKS_IN]→(dept) DELETE r
```

Table

Code

Deleted 1 relationship, completed after 32 ms.

- Deleting relationship is as simple as deleting nodes. Use the MATCH statement to match the relationships you want to delete.
- You can delete one or many relationships or all relationships by using one statement.

Delete All Relationships

```
neo4j$ MATCH (n)-[r]-() DELETE r;
```

Table

Code

Deleted 1 relationship, completed after 14 ms.

Delete All Nodes and Relationships

```
neo4j$ MATCH (n) DETACH DELETE n;
```

Task

1. Create a new project, then create a new DBMS and set it up (Specufy SocialDB as the database name), then start the database and open Neo4j Browser.
2. Add users as nodes containing name, email, and age:
   a. user : “Ahmed” , email: “ahmed@example.com”, age: 25.
   b. user: “Mona”, email: “mona@example.com”, age: 22.
3. Add posts as nodes containing content and publication date:
   content : “Hello World” , date: “2025-03-27”
4. Add comments as nodes containing the comment text and comment date:
   text: “Nice post!”, date: “2025-03-27”
5. Link the user to their post (the user wrote this post).
6. Get another user to like the same post.
7. Link the comment to the user and post.
8. Delete all likes.

Thank You